Project: JobTrackr

Core Functionality

Application Management

- Add, edit, delete job applications
- Fields: company, role, salary range, job URL, date applied, source (LinkedIn, Naukri, Referral etc.), notes
- Status pipeline: Saved → Applied → Interview → Offer → Rejected

Kanban Board

- Drag and drop cards across status columns
- Quick edit inline

Dashboard & Stats

- Total applied, response rate, interview conversion rate
- Applications over time (chart)
- Best performing sources

Follow-up System

- Set follow-up reminder date per application
- Notification/badge when follow-up is due

Monorepo (single repo, two folders)

jobtrackr/

├── jobtrackr-api/ ← NestJS

└── jobtrackr-web/ ← Next.js

AI Features (the interesting part)

1. Resume-to-JD Match Scorer Paste a job description → AI scores how well your resume matches it (0–100) and tells you what's missing. Genuinely useful for you right now, impressive in interviews.

2. Cold Email / Follow-up Generator One click → AI drafts a follow-up email based on company name, role, and days since applying. You edit and send.

3. Interview Prep Generator Based on the job title and description → AI generates 5 likely interview questions with suggested answer frameworks.

4. Application Auto-fill (stretch goal) Paste a job URL → AI extracts company, role, and key requirements automatically.

Tech Stack

Layer	Technology	Why
Frontend	Next.js 14, TypeScript	Your strength
Styling	Tailwind CSS, shadcn/ui	Your strength
Server state	React Query	Honest resume claim
Backend	NestJS, REST, Swagger	What you want to learn
Database	PostgreSQL + Prisma ORM	Easier than raw SQL to start
Auth	JWT (NestJS) + NextAuth	You've done this
AI	Claude API (Anthropic)	Better than OpenAI for text tasks
Hosting	Vercel (FE) + Railway (BE)	Free tier, simple

DB Schema (simple version to start)

User
- id, email, name, password_hash, created_at

Application
- id, user_id, company, role, status
- salary_min, salary_max, currency
- job_url, source, notes
- applied_date, followup_date
- created_at, updated_at

AILog (track AI usage per application)
- id, application_id, type (match_score | email | prep)
- input, output, created_at

API Endpoints (NestJS)

Auth
POST /auth/register
POST /auth/login
POST /auth/refresh

Applications
GET /applications — list all (with filters)
POST /applications — create
GET /applications/:id — single
PATCH /applications/:id — update
DELETE /applications/:id — delete

AI
POST /ai/match-score — resume vs JD
POST /ai/follow-up-email — generate email
POST /ai/interview-prep — generate questions

Build Order (week by week)

Week 1 — Backend foundation NestJS setup, Postgres + Prisma, auth (register/login/JWT), applications CRUD, Swagger docs working

Week 2 — Frontend Next.js setup, auth flow, application list + form, Kanban board with drag and drop, React Query for all API calls

Week 3 — AI features + Polish Claude API integration, match scorer, email generator, interview prep, dashboard stats, deploy both ends

Zero Cost Setup

- **Frontend:** Vercel — free
- **Backend:** Fly.io — free tier
- **Database:** Supabase — free tier
- **AI:** Claude API — $5 free credit lasts months for personal use

Recommended Zero Cost Stack

Layer	Service	Cost
Frontend	Vercel	₹0
Backend	Fly.io	₹0

Database	Supabase	₹0
AI	Claude API (free credits)	₹0 for ~3 months
Total		**₹0/month to start**

Priority 1 — Auth flow Decide now because everything else is built on top of it.

- Simple JWT (access token only) is fine to start
- Refresh tokens add complexity — skip for now, add later
- **Google OAuth via Supabase** — I'd recommend adding this. One click login looks professional, Supabase handles the complexity, and it's a talking point in interviews. Not hard to add.

Priority 2 — Resume storage for JD matcher This affects your DB schema which you haven't pushed yet — so decide now.

- **Option A:** User uploads PDF every time they use the matcher — simpler, no storage needed
- **Option B:** User saves resume text once in their profile — better UX, just add a `resumeText` field to the `User` table

Option B is one extra DB field. I'd go with that.

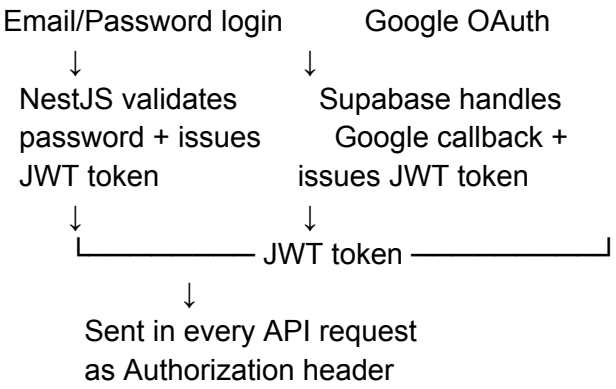
Priority 3 — AI rate limiting Affects backend architecture. Simple solution — add an `aiCallsToday` counter on the User table. Reset daily. Cap at 10 calls/day. Prevents accidental cost spikes if you share the app publicly.

Priority 4 — Auto-deploy from day one Set up Vercel (frontend) and Fly.io (backend) connected to your GitHub repo before you start coding. That way every push deploys automatically. Interviewer can see a live URL from week 1.

Everything else — dark mode, drag-and-drop library, branding — can be decided during build without consequences.

Priority 1: Let's have a simple JWT for now. refresh tokens skip for now. Let's use Google OAUTH
Priority 2: Lets go with option B
Priority 3: lets go with your recommended flow
Priority 4: go with auto-deploy

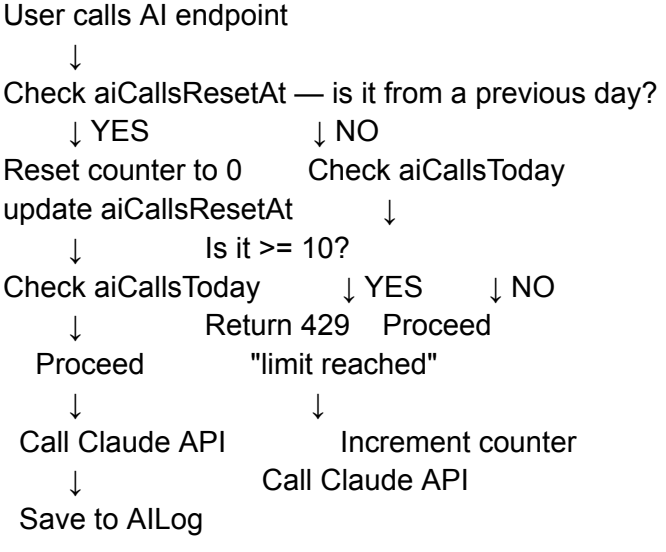
Auth Flow



Both paths end in the same JWT — your frontend doesn't need to care which method was used.

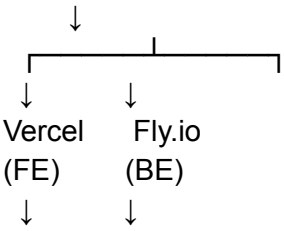
AI Rate Limit Logic

Simple flow inside your AI service:



Auto-deploy Setup Plan

You'll do this once after `git init`:



Both Vercel and Fly.io connect directly to your GitHub repo — you just push, they build and deploy automatically. We'll set this up before writing any feature code.

Final Architecture Summary

Decision	Choice
Auth	JWT + Google OAuth via Supabase
Refresh tokens	Skip for now
Resume storage	Saved as text in User profile
AI rate limit	10 calls/day, resets midnight
Deploy	Auto-deploy — Vercel + Fly.io
DB	Supabase (Postgres)
ORM	Prisma
AI Model	Claude Haiku (fast + cheap)